Clustering, Classification and Optimisation

Richard Willis

April 28, 2025

Outline

1. Intro
2. K-means
3. Naive Bayes Classifier
4. Optimisation

Intro

We will go through the following Google Colab worksheets:

1. K-means, an example of unsupervised clustering
2. Naive Bayes, an example of supervised classification
3. Optimisation, an introduction to gradient descent

Please check that you can assess and run them!

K-means

- Unsupervised learning [1] means that we have a dataset, which is unlabelled, and we wish to discover patterns and insights within it.
- *Clustering* algorithms are one example of unsupervised learning algorithms, which find groups of points that are more similar to each other than those in other groups, or clusters.
- The K-means algorithm is the most simple clustering algorithm, and it works by partitioning the feature space into k clusters. Amazing visualisation blog.

[1] https://cloud.google.com/discover/what-is-unsupervised-learning

Algorithm

Steps

Fitting

- Randomly create k cluster centres
- Loop:
  1. Compute the distance of each point to each cluster centre
  2. Assign each point to its closest cluster centre
  3. Update the cluster centres to be the mean of the points assigned to them

Inference

1. Compute the distances to each cluster centre.
2. Assign the data point to the closest cluster centre.

Details

We can either continue until the cluster centres do not update (this will happen when no point has changed its assigned cluster), or terminate early, as long as no cluster centre moved by more than a tolerance value.

Naive Bayes Classifier

Used with categorical data for supervised learning

Bayes theorem

$$P(\text{Hypothesis} \mid \text{Data}) = \frac{P(\text{Data} \mid \text{Hypothesis}) \times P(\text{Hypothesis})}{P(\text{Data})}$$

- $P(\text{Hypothesis})$ is the prior probability of the hypothesis before observing the data.
- $P(\text{Data} \mid \text{Hypothesis})$ is the likelihood of the data given that the hypothesis is true.
- $P(\text{Hypothesis} \mid \text{Data})$ is the posterior probability of the hypothesis given the data
- $P(\text{Data})$ is the probability of observing the data under all possible hypotheses.

Example (1)

Weather	Temperature	Wind	Tennis
Sun	Hot	Strong	No
Rain	Cold	Mild	No
Cloudy	Cold	Mild	Yes
Sun	Hot	Mild	Yes

- Prior : $P(tennis = yes) = \frac{2}{4}$
- Conditional Probability : $P(wind = mild|tennis = no) = \frac{1}{2}$
- *Naive* because we assume that the features are *conditionally independent*, so that

$$P(weather, temperature, wind|tennis) = P(weather|tennis)P(temperature|tennis)P(wind|tennis)$$

Without the assumption of conditional independence, to work out $P(sun, hot, mild|yes)$ we would need to have observed this *exact* outcome before. We therefore tend to use Naive Bayes even when this assumption is violated.

Conditional independence

The features are not *independent*: the weather is causally related to the temperature.
Are the features *conditionally independent*? Suppose I never play tennis when it is both raining and windy. Then $P(rain|yes)$ and $P(strong|yes)$ are not independent: if one is true then the other must be false.

What might conditional independence look like?

Weight	Likes baths	Animal
Light	No	Cat
Heavy	Yes	Dog
Heavy	No	Cat

Cats typically hate baths. But fat (heavy) cats are just as likely to hate baths as normal cats. Therefore, conditional on the animal, knowing the weight does not influence whether they enjoy bathing.

Example (2)

Table: Do I play tennis?

Weather	Temperature	Wind	Tennis
Sun	Hot	Strong	No
Rain	Cold	Mild	No
Cloudy	Cold	Mild	Yes
Sun	Hot	Mild	Yes
Sun	Cold	Mild	?

$$P(tennis = yes|data) \propto P(yes)P(sun|yes)P(cold|yes)P(mild|yes)$$
$$= \frac{2}{4} \times \frac{1}{2} \times \frac{1}{2} \times \frac{2}{2} = \frac{1}{8}$$

$$P(tennis = no|data) \propto P(no)P(sun|no)P(cold|no)P(mild|no)$$
$$= \frac{2}{4} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{16}$$

Therefore, $P(yes|data) = \frac{P(yes|data)}{P(yes|data)+P(no|data)} = \frac{\frac{1}{12}}{\frac{1}{8}+\frac{1}{16}} = \frac{2}{3}$

Algorithm

Steps

Fitting

1. Compute the prior
2. Compute the feature likelihoods

Inference

1. For each class, multiply the prior probability by the likelihood, which is the product of the conditional feature probabilities
2. Select the class with the highest probability

Details

We use log probabilities, to avoid floating point precision errors.

Optimisation

In machine learning, we often work with cost functions where analytical solutions are impractical or impossible. Instead, we rely on numerical approaches to find optimal parameters that minimise the cost function.

Gradient descent

Can be used for any parameterised function, for example, a neural network. Where $\boldsymbol{\theta}$ represents the parameters of our neural network and $f(\boldsymbol{\theta})$ represents the cost function to be minimised (the training loss).

$$\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \alpha \nabla f(\boldsymbol{\theta}_n)$$

Starting from an initial guess, $\boldsymbol{\theta}_0$, we repeatedly takes steps in the direction of steepest descent until we find a minimum. This method is sensitive to both the initial guess, and the hyper-parameter α that governs the step size. Failures include failing to converge, or converging to a local minimum.

Gradient descent example

In the workbook we will use single independent variable linear regression. Coordinates represent the values of the parameters $\boldsymbol{\theta}$.

$$h_\theta(x) = \theta_0 + \theta_1 \cdot x$$

Cost function

A standard cost function, $f(\boldsymbol{\theta})$, is the mean squared error:

$$f(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradients

$$\frac{\partial f}{\partial \theta_0} = \frac{1}{n} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)}) \quad \text{and} \quad \frac{\partial f}{\partial \theta_1} = \frac{1}{n} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

First order iterative method

- Finds the gradient at the current point: this represents the direction of steepest ascent.
- Best when we have analytical forms for the gradient, though we can numerically approximate the gradient if we do not know the analytical form.

Trade-off: First-order methods require computing fewer derivatives (computationally efficient per iteration) but typically need more iterations to converge compared to higher-order methods that incorporate curvature.